

LogBase: A Large-Scale Benchmark for Semantic Log Parsing

CHENBO ZHANG, Fudan University, China

WENYING XU, Fudan University, China

JINBU LIU, Alibaba Group, China

LU ZHANG, Fudan University, China

GUIYANG LIU, Alibaba Group, China

JIHONG GUAN, Tongji University, China

QI ZHOU, Alibaba Group, China

SHUIGENG ZHOU*, Fudan University, China

Logs generated by large-scale software systems contain a huge amount of useful information. As the first step of automated log analysis, log parsing has been extensively studied. General log parsing techniques focus on identifying static templates from raw logs, but overlook the more important semantics implied in dynamic log parameters. With the popularity of Artificial Intelligence for IT Operations (AIOps), traditional log parsing methods no longer meet the requirements of various downstream tasks. Researchers are now exploring the next generation of log parsing techniques, i.e., *semantic log parsing*, to identify both log templates and semantics in log parameters. However, the absence of semantic annotations in existing datasets hinders the training and evaluation of semantic log parsers, thereby stalling the progress of semantic log parsing.

To fill this gap and advance the field of semantic log parsing, we construct **LogBase**, the first semantic log parsing benchmark dataset. LogBase consists of logs from 130 popular open-source projects, containing 85,300 semantically annotated log templates, surpassing existing datasets in both log source diversity and template richness. To build Logbase, we develop the framework **GenLog** for constructing semantic log parsing datasets. GenLog mines log template-parameter-context triplets from popular open-source repositories on GitHub, and uses chain-of-thought (CoT) techniques with large language models (LLMs) to generate high-quality logs. Meanwhile, GenLog employs human feedback to improve the quality of the generated data and ensure its reliability. GenLog is highly automated and cost-effective, enabling researchers to easily and efficiently construct semantic log parsing datasets. Furthermore, we also design a set of comprehensive evaluation metrics for LogBase, including general log parser metrics and the metrics specifically for semantic log parsers and LLM-based parsers.

With LogBase, we extensively evaluate 15 existing log parsers, revealing their true performance in complex scenarios. We believe that this work provides researchers with valuable data, reliable tools, and insightful findings to support and guide the future research of semantic log parsing.

CCS Concepts: • **Software and its engineering** → **Maintaining software**.

*Corresponding author.

Authors' Contact Information: [Chenbo Zhang](#), Fudan University, Shanghai, China, cbzhang21@m.fudan.edu.cn; [Wenyong Xu](#), Fudan University, Shanghai, China, xuwy22@m.fudan.edu.cn; [Jinbu Liu](#), Alibaba Group, Hangzhou, China, liujinbu.ljb@alibaba-inc.com; [Lu Zhang](#), Fudan University, Shanghai, China, l_zhang19@fudan.edu.cn; [Guiyang Liu](#), Alibaba Group, Hangzhou, China, wuming.lgy@alibaba-inc.com; [Jihong Guan](#), Tongji University, Shanghai, China, jhguan@tongji.edu.cn; [Qi Zhou](#), Alibaba Group, Hangzhou, China, jackson.zhouq@alibaba-inc.com; [Shuigeng Zhou](#), Fudan University, Shanghai, China, sgzhou@fudan.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2025/7-ARTISSTA092

<https://doi.org/10.1145/3728969>

Additional Key Words and Phrases: Benchmark, Semantic log parsing, AIOps, Large language model.

ACM Reference Format:

Chenbo Zhang, Wenying Xu, Jinbu Liu, Lu Zhang, Guiyang Liu, Jihong Guan, Qi Zhou, and Shuigeng Zhou. 2025. LogBase: A Large-Scale Benchmark for Semantic Log Parsing. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA092 (July 2025), 22 pages. <https://doi.org/10.1145/3728969>

1 Introduction

Logs of software systems record important events and states during operation and play a vital role in modern software development and maintenance. By analyzing these logs, software engineers are able to perform tasks such as anomaly detection [9, 15, 35, 63, 64, 66], root cause analysis [3, 18, 33], and failure prediction [7, 65]. With the popularity of large-scale software systems, the volume of logs they produce has been drastically increasing. Taking Alibaba Inc.'s cloud computing systems as an example, these systems generate about 30 to 50 GB of logs per hour [28]. Such a massive scale of log data poses an urgent need for automated log analysis to replace traditional manual analysis. *Log parsing*, the first step in automated log analysis, transforms semi-structured logs into structured ones suitable for downstream tasks. Specifically, the log parsers distinguish between static log templates and dynamic log parameters in the raw logs. For instance, given the log message “Connection completed. Connection status: 0”, “Connection completed. Connection status: <*>” is the template, which is set by developers at the time of programming and usually remains unchanged, and “0” is the log parameter, which may change with the running state of the program.

With the rapid development of artificial intelligence (AI) for IT operations (AIOps), downstream tasks increasingly demand high-quality log data. Such tasks, including anomaly detection and system performance analysis, rely on upstream log parsers to provide semantically rich information. In this context, general log parsing techniques, while capable of identifying basic elements such as log templates and parameters, fail to identify the specific meanings of these parameters (e.g. simplifying them to a wildcard <*>), thus do not take full advantage of these data, as they ignore the more important semantics implied in the log parameters.

In contrast, *semantic log parsing* represents a significant advancement in log parsing [20, 32, 57]. It can not only recognize the templates and parameters in logs but also unveil the specific semantics of these parameters, greatly enhancing the utility of log data. For example, given the log message: “Connection completed. Connection status: 0”, a general log parser can only extract “Connection completed. Connection status: <*>” as the template and treat “0” as a constant. However, a semantic log parser would identify “0” as a status code, which enables software engineers to select a specific algorithm for anomaly detection from logs based on the specific types of log parameters, thereby improving the accuracy of fault diagnosis and the effectiveness of system maintenance. Semantic log parsing techniques transform log data from passive records into dynamic resources that can support complex decisions and operations. The shift from general log parsing to semantic log parsing significantly boosts the capability of AIOps systems to handle complex data, paving the road to more effective and intelligent operations.

It is well-known that in AI and its various application areas, benchmark datasets are crucial to model training and evaluation. However, existing log parsing benchmarks [22, 67] have two major limitations. First, existing benchmarks are mainly designed for general log parsing, and are unsuitable for training and evaluating more advanced semantic log parsers [32]. Second, these benchmarks fall short in terms of log source diversity and template richness, deviating significantly from real-world log analysis environments. Many parsers performing well on these benchmarks show significant performance drops in complex real-world scenarios, indicating that these benchmarks cannot be used to reliably evaluate the actual performance of parsers. [12, 27, 44]

To support the research of semantic log parsing, we construct a new log benchmark dataset **LogBase**, which overcomes the limitations of existing datasets. To the best of our knowledge, it is the first large-scale semantic log parsing benchmark dataset. Logs in LogBase are from 130 popular open-source repositories on GitHub, containing 85,300 semantically annotated log templates, largely surpassing the scale of the largest existing dataset LogHub-2.0 [22], specifically, 130 v.s. 14 in sources, and 85,300 v.s. 3,488 in log templates. As a benchmark for semantic log parsing, LogBase features a hierarchical log semantics taxonomy, including eight coarse-grained semantic categories and sixteen fine-grained semantic categories, allowing for a more comprehensive and accurate evaluation of semantic log parsers.

To build LogBase, we develop the **GenLog** framework for efficiently constructing semantic log parsing datasets. Concretely, GenLog's dataset construction process consists of two phases: data collection and human-enhanced log generation. During the data collection phase, we develop a crawler program to automatically download source code from the most popular open-source repositories on GitHub. We then use JavaParser [48] (ast library in Python) to analyze the obtained source code, transforming them into Abstract Syntax Trees (ASTs). By traversing the ASTs, we extract all log recording statements, log templates, parameters, and their contextual codes to build a large number of log template-parameter-context triplets for large language models (LLMs) [1] to generate log data. In the subsequent human-enhanced log generation phase, we employ chain-of-thought (CoT) [54] techniques combined with LLMs to generate semantic templates and high-quality logs from these template-parameter-context triplets. To mitigate the impact of hallucinations of the LLMs on the generated data, we perform manual reviews of the generated content and use human feedback to help the LLMs improve the quality of the generated data. This ensures that the generated data accurately reflects logs in real-world environments. Notably, the entire construction process is automated, significantly reducing the cost associated with log collection and template annotation, which makes the construction of log parsing datasets easy, efficient, and cost-effective.

Furthermore, we design a set of comprehensive evaluation metrics, including metrics assessing both general and semantic log parsing performance, and cost metrics specifically for LLM-based parsers [21, 60], helping explore the performance-cost balance of parsers in practical scenarios.

With LogBase and the metrics, we evaluate various existing log parsing techniques, including general log parsers and semantic log parsers. The results reveal the true performance of current parsers in complex scenarios.

Contributions of this paper are as follows:

- We construct **LogBase**, to the best of our knowledge, the first and the largest benchmark dataset for semantic log parsing to date. It consists of logs from 130 popular open-source projects on GitHub, and contains 85,300 semantically annotated templates. LogBase features a hierarchical semantic taxonomy with eight coarse-grained and sixteen fine-grained categories, enabling a fine evaluation of semantic log parsers.
- We propose the **GenLog** framework for constructing semantic log parsing datasets. GenLog automates the dataset construction process, including data collection and log generation, and leverages human review and feedback to ensure the reliability of the generated logs. This approach significantly reduces the cost and errors associated with data generation, enabling easy, efficient, cheap, scalable, and accurate construction of large-scale log parsing datasets directly from source code, to provide high-quality and semantically rich log data for advanced log analysis tasks.
- We introduce a set of comprehensive performance metrics for semantic log parsing, including cost metrics for LLM-based parsers, which enable a thorough evaluation of semantic log parsers in practical scenarios.

- We conduct extensive experiments on LogBase to evaluate 15 existing log parsers, covering both general log parsing and semantic log parsing. Our experiments reveal the true performance of existing methods, and highlight their strengths and weaknesses. These results provide helpful insights into the applications and further development of these methods.

The rest of this paper is organized as follows: Sec. 2 presents the background and motivation of this study. Sec. 3 introduces the details of dataset construction, i.e., the GenLog framework. Sec. 4 and Sec. 5 are experimental design and results, respectively. Sec. 6 is discussions. Sec. 7 briefs the threats to validity. Finally, Sec. 8 concludes this paper.

2 Background and Motivation

2.1 Existing Log Parsing Techniques

Existing log parsing techniques can be divided into *general log parsing* and *semantic log parsing* based on their objectives. The goal of general log parsing is to identify static log templates from raw logs and replace dynamic log parameters with wildcards like “<*>”. In contrast, semantic log parsing enhances general log parsing by not only identifying log templates but also recognizing the semantics of dynamic log parameters with semantic labels. However, currently semantic log parsing is still in its infant stage, only a few works have been reported.

2.1.1 General Log Parsing. Based on the employed techniques, existing general log parsers can be categorized into unsupervised parsers, supervised parsers, and large language model (LLM)-based parsers. Unsupervised parsers can be further divided into frequency-based, similarity-based, and heuristics-based. Frequency-based parsers [6, 40, 50, 51] mine frequent itemsets from logs to generate templates. Similarity-based parsers [11, 16, 39, 46] cluster logs by similarity, regarding that each cluster implies a template. Heuristics-based parsers [8, 17, 23, 37, 38, 45, 61] use algorithms like the longest common subsequence and parsing trees to differentiate templates and parameters. Supervised parsers [29, 32, 34, 53] utilize natural language processing (NLP) techniques with architectures like Bi-LSTM [14] and Transformer [52], can achieve high accuracy after being trained on large datasets, though having slow parsing speeds. Recently, LLM-based parsers [19, 21, 58–60], such as those using ChatGPT [1], have been developed. These models, pre-trained on massive internet data, can distinguish log templates and parameters by text semantics understanding. By in-context learning (ICL), they can be quickly adapted to new log patterns by using relevant examples, and perform well in real dynamic log analysis scenarios with enhanced flexibility and efficiency.

2.1.2 Semantic Log Parsing. VALB [32] is the first semantic log parser, transforming semantic log parsing into a sequence labeling task. It utilizes a BiLSTM-CNN-CRF structure to identify the semantic category of each log token. SemParser [20] introduces an end-to-end semantic miner and joint parser, leveraging domain knowledge to identify semantics in logs. Following VALB’s setting, LogPTR [57] constructs an end-to-end semantic log parser based on the pointer network.

2.2 Existing Log Parsing Benchmarks

LogHub [67], developed by the LogPAI team, is a widely used benchmark dataset for log parsing. It collects logs from 16 different sources, including distributed systems, supercomputers, and operating systems. Annotators manually annotated templates from 2,000 randomly sampled log messages for each source. In total, LogHub comprises 1,313 log templates and 32,000 annotated logs.

LogHub-2.0 [22], an expanded version of LogHub, is currently the largest log parsing benchmark dataset. While the number of log sources has been reduced from 16 to 14, the LogPAI team has conducted extensive template labeling on the logs from these 14 sources. As a result, LogHub-2.0 contains 3,488 log templates and 50,416,618 annotated logs.

2.3 Motivation

Despite the promising performance of many log parsers on LogHub and LogHub-2.0, such as LogPPT [29] achieves over 90% group accuracy and parsing accuracy on LogHub, they show significant performance drop in real-world complex log analysis tasks [12, 27, 44]. This suggests that existing benchmark datasets fail to accurately evaluate the true performance of log parsers. Additionally, existing datasets are designed for general log parsing, thus unsuitable for training and evaluating semantic log parsers. To address these issues, we try to construct a new large-scale benchmark dataset for semantic log parsing with the goals as follows:

2.3.1 Supporting Semantic Log Parsing. With the advancement of AIOps, general log parsing is no longer adequate to support downstream tasks. Semantic log parsing can identify log templates and recognize log parameters' semantics, thus greatly helping downstream AIOps tasks [20, 32]. For instance, in log anomaly detection, the semantics of log parameters enable the selection of suitable anomaly detection algorithms, leading to more accurate results. However, existing datasets lack semantic annotations, hindering the training and evaluation of semantic log parsers. To advance log parsing and its applications, semantically annotated logs are essential.

2.3.2 Enhancing Diversity of Log Templates and Sources. Real-world logs are complex, covering various scenarios from operating systems to distributed systems and applications. Current datasets are primarily derived from operating system-level logs, such as LogHub-2.0, which contains only 14 log sources and 3,488 log templates, thus lacking diversity. Real-world AIOps systems require handling logs from various templates across both application and system levels. This gap makes many log parsers perform poorly in complex environments [12, 27], despite working well on LogHub-2.0. Therefore, to narrow the gap between academic benchmarks and real-world application scenarios, it is essential to enhance the diversity of log sources and templates as much as possible.

2.3.3 Establishing Comprehensive Evaluation Metrics. Existing datasets, designed for general log parsing, are unsuitable for evaluating semantic log parsers. To construct benchmarks for semantic log parsing, new evaluation metrics are necessary. Additionally, LLM-based parsers are becoming popular for their high performance. However, they are also expensive. In practical scenarios, it is important to balance cost and performance, but existing methods do not consider model cost. Thus, we also need to consider some metrics specifically for evaluating the cost of LLM-based parsers.

3 Benchmark Construction

3.1 Framework

One challenge of constructing large-scale log datasets is how to obtain raw logs. Unlike images and natural language texts, logs usually contain sensitive information, which makes them rarely publicly available online. This hinders the progress of log analysis research. For example, the popular log datasets like LogHub and the more recent LogHub-2.0, have not updated their raw

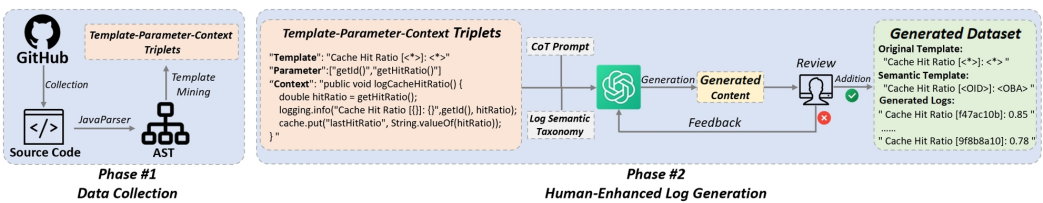


Fig. 1. The framework of GenLog.

logs for several years due to the acquisition challenge. As software continuously evolves, logs from several years ago substantially differ from those of current environments, making such benchmark datasets inadequate to reflect real-world log analysis scenarios.

To overcome this challenge, we develop **GenLog**, a framework for efficiently constructing semantic log parsing datasets. By acquiring raw data from popular open-source repositories and using LLMs to generate high-quality logs and their annotations, GenLog can automatically create up-to-date log datasets from scratch, effectively addressing the data scarcity issue.

Fig. 1 shows the framework of GenLog, whose data generation process consists of two phases: *data collection* and *human-enhanced log generation*. In the data collection phase, we use web crawling techniques to automatically download the source code from the popular open-source repositories on GitHub and utilize JavaParser [48] to automatically mine log template-parameter-context triplets from the source code. In the human-enhanced log generation phase, we employ online LLMs to semantically annotate the log templates and generate high-quality logs based on the mined log template-parameter-context triplets. Simultaneously, human feedback ensures the reliability of the LLM-generated content. Experienced engineers review the LLM’s output, manually re-annotate inaccurate results, and use their feedback to help the LLM generate more reliable log data.

Table 1. The hierarchical log semantic taxonomy in LogBase, including coarse-grained and fine-grained categories, with description and log proportion of each fine-grained category.

Coarse-grained Categories	Fine-grained Categories	Abbrev.	Description	Proportion
Object	Object ID	OID	Identification information of an object.	10.7%
	Object Name	OBN	Name of an object.	20.7%
Location	Location Indicator	LOI	Location information of an object.	12.5%
Status	Type Indicator	TID	Type information of an object or an action.	1.9%
	Status Indicator	SID	Status code of an object or an action.	4.5%
Time	Time Point of Action	TPA	Specific time an action occurs.	1.1%
Count	Object Amount	OBA	Amount of objects.	13.5%
	Computing Resources	CRS	Information of computing resources.	2.0%
	Time Duration of Action	TDA	Time duration of an action.	3.2%
Content	Data Content	DCT	Data content in logs.	9.9%
Error	Error and Exception	ERR	Information of errors and exceptions.	10.6%
Others	Command	CMD	Information of executed commands.	0.9%
	Object Order	OBO	Order information of an object.	2.9%
	Interval	INR	Information of data intervals.	0.1%
	Version Information	VER	Details about the version of an object.	0.8%
	Other Parameters	OTP	Other information.	4.7%

3.2 Hierarchical Log Semantic Taxonomy

A proper log category system is essential for log benchmark datasets. Although VALB [32] defines 10 semantic categories based on LogHub, these categories are too coarse for precise semantic log parsing. For instance, it does not specifically categorize frequently occurring and significant exception information in logs, instead, it groups them under the “Others” class, leading to fine semantic information loss.

To investigate this issue more deeply, we revisit the VALB’s category system and apply it to annotating the more diverse LogHub-2.0 dataset. We invite experienced engineers to evaluate the accuracy and suitability of the labeling. The results show that many critical parameters are classified as “Other”, including runtime errors (which should belong to the “Error” category). These log parameters carry essential semantics for downstream tasks. This suggests that the VALB’s category system is too coarse and unsuitable for complex, large-scale log datasets. The primary reason lies in VALB’s original design, which is tailored to LogHub-2K [67], a relatively

straightforward dataset with limited template richness and complexity. Consequently, it struggles to accurately define the diverse log parameters encountered in real-world scenarios.

To address this issue, we propose a new semantic category system designed for large-scale log parsing datasets and real-world environments. Specifically:

- (1) We start by collecting log parameters and templates that cannot be accurately annotated in our previous tests, and enlist our experienced engineers to define new semantic categories for these parameters.
- (2) We then integrate these newly defined categories into the existing system, and conduct a review to ensure clarity and consistency. To maintain simplicity, we remove semantically excessively fine categories and establish parent categories for similar types (e.g. defining “Object” as a parent category for “Object ID” and “Object Name”).
- (3) We sample 2,000 logs from real production environments and annotate them using the new semantic category system. After annotating, we review the results, identifying the parameters and templates categorized as “Other”, which cannot be accurately defined.
- (4) Step 1, 2, and 3 above are repeated until the proportion of entries classified as “Other” in the results falls below 5%.

With this iterative process of annotation, integration, and review, we establish a new hierarchical log semantic taxonomy. Tests on real-world data show that this taxonomy can accurately define the vast majority of logs across both complex benchmarks and real-world environments.

As presented in Tab. 1, our taxonomy is organized into two levels. The first level encompasses 8 coarse-grained semantic categories of log parameters: *Object*, *Location*, *Status*, *Time*, *Count*, *Content*, *Error*, and *Others*. Each of these categories represents a coarse type of parameter. At the second level, these categories are further refined into 16 fine-grained semantic categories to provide more detailed insights. For example, the coarse-grained category “*Object*” is divided into two fine-grained categories: “Object ID (OID)” and “Object Name (OBN)”. For some coarse-grained categories, such as “*Location*”, no further subdivision is made due to the limited content they encompass.

3.3 Data Collection

GitHub, the world’s largest open-source project hosting site, contains the source code of many widely used software applications. We develop a web crawler to automatically download the 110 most-starred Java repositories and the 20 most-starred Python repositories. These high-quality repositories are maintained and updated by reliable organizations and cover various real-world software domains, such as application development, data management, and network communication, which are closely consistent with the real log analysis scenarios.

After collecting the source code from various open-source repositories, running the code to gather raw logs not only is impractical due to the high run-time and debugging costs, but also cannot capture all possible log types. To circumvent this problem, we leverage the strong generative and generalization power of LLMs, which have achieved great success in areas like code generation [10, 30, 41], document generation [4, 36], and unit test generation [26, 31, 62]. Instead, we mine log template-parameter-context triplets from the source code, and use them to guide LLMs in generating high-quality log data, ensuring high coverage of all possible log types and templates.

To systematically mine log templates, we utilize the Abstract Syntax Tree (AST) because it provides a detailed and hierarchical representation of the source code, making it easy to analyze and extract specific patterns. We first use JavaParser (ast library in Python) to parse the collected source code into an AST, where each node represents a syntactic construct, such as a method call or a variable declaration. By traversing the AST, we examine each node to accurately identify logging statements. During the traversal, we search for method call expressions that correspond to logging

functions (e.g. `logging.debug()`). For each found logging statement, we extract the log template, its parameters, and their contextual code, and convert them into log template-parameter-context triplets. To ensure the quality of the mined data, we exclude logging statements that consist solely of punctuation marks or contain only log parameters without meaningful templates.

Finally, we obtain all the log template-parameter-context triplets from the projects without running the source code, providing a solid foundation for LLMs to generate logs subsequently.

3.4 Human-Enhanced Log Generation

After obtaining the log template-parameter-context triplets, we use a powerful online LLM to semantically annotate these templates and generate high-quality log data. This is because online LLMs are pre-trained on vast amounts of data, including code, and have internet search capabilities. These abilities allow them to effectively understand the meaning of log parameters based on their own knowledge and online information, perform semantic classification, and generate logs. To ensure the reliability of the content generated by the LLM, our team members review the output and provide feedback to enhance the quality of the generated content.

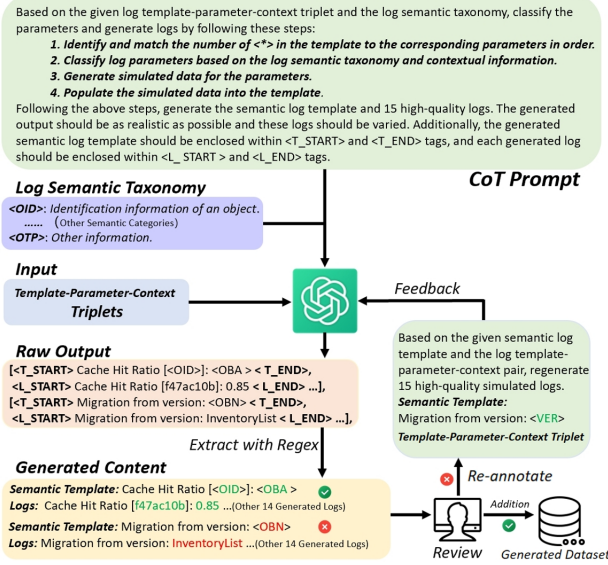


Fig. 2. An example of human-enhanced log generation.

Additionally, to guarantee the diversity of the generated logs, we explicitly specify that each log generated should be as different as possible from the others. This instruction helps in producing a large variety of logs to reflect different potential scenarios.

One challenging issue is that LLMs have *hallucinations*, meaning the generated content could be unreliable or incorrect. Although the internet search capabilities of online LLMs and our extensive contextual reference information can greatly reduce the likelihood of hallucination, the hallucination phenomenon cannot be avoided. To further ensure the reliability of the generated logs and their similarity to real-world logs, we employ manual review and feedback.

Specifically, four experienced engineers from our team review each batch of logs generated by the LLM. They first verify the accuracy of the semantic annotations of the log templates. If the annotations are correct, they proceed to check the authenticity of the synthesized logs. logs

Concretely, as shown in Fig. 2, we carefully design prompts for the log generation process. First, we provide the LLM (e.g. ChatGPT) with the log semantic taxonomy as a reference, then define the goals to ensure that it understands the task. Next, we use chain-of-thought (CoT) [54] to divide the generation task into smaller and manageable steps, guiding the LLM to complete the task sequentially. This process includes: 1) Establishing the correspondence between log templates and their corresponding parameters. 2) Classifying log parameters based on the log semantic taxonomy and contextual information. 3) Generating high-quality annotation for each log parameter based on its semantics. 4) Populating the log templates with the generated data to produce the final results.

that pass both checks are added to the final dataset. If any output fails either check, the engineers manually annotate the semantics and use these annotations as additional reference information. These augmented log template-parameter-context triplets are then re-entered into the LLM for regeneration. This *generate-review-feedback* cycle repeats until all generated logs pass the checks and are added to the final dataset.

In the entire generation process, 68% of the templates can produce correct semantic annotations and synthesized logs directly from the LLM without human correction. For 93% of templates, the correct result can be achieved with no more than one human adjustment. Furthermore, 98% of templates yield the correct result after no more than two human corrections. For the remaining 2% of templates, achieving accurate results proves challenging due to the limited semantic information contained in these templates, making them difficult for the LLM to interpret. For these cases, our engineers manually annotate and synthesize the logs based on their expertise.

3.5 Results

With the GenLog framework, we construct the LogBase dataset, the first semantic log parsing benchmark dataset, encompassing 130 various log sources and 85,300 log templates. This makes it significantly larger and more complex than the existing largest dataset, LogHub-2.0 [22], with over 9.3 and 24.5 times larger in terms of log sources and templates, respectively. Unlike the existing datasets [22, 67], LogBase is equipped with semantic annotations for each log, positioning it as the first dataset specifically designed for training and evaluating semantic log parsing models. This unique feature, along with its unprecedented scale and diversity in templates, can play a significant role in advancing the semantic log parsing field.

4 Experimental Design

4.1 Research Questions

- **RQ1:** What are the differences and advantages of LogBase compared to the existing benchmarks?
- **RQ2:** How do existing parsers perform in general log parsing on LogBase?
- **RQ3:** How do existing parsers perform in semantic log parsing on LogBase?
- **RQ4:** Are the synthetic logs valid?

4.2 Experimental Settings

We evaluate 15 existing log parsers, including general log parsers and semantic log parsers. For general log parsers, we select unsupervised parsers [6, 8, 16, 17, 23, 37–40, 45, 51], supervised parsers [29, 32], and LLM-based parsers [21, 60]. Some methods [19, 58, 59] are not open-source, so we are unable to evaluate them. As for semantic log parsers, we do not evaluate SemParser [20] and LogPTR [57] because they are not open-sourced and lack reproducibility details. Instead, we evaluate VALB [32], and use the classic named entity recognition (NER) method Bert-NER [25] as a baseline to do semantic log parsing for comparison. For most compared methods, we use their original implementation code and hyperparameter settings from the corresponding papers. For the other methods, we use their open-source code or implement them following the original papers.

We conduct experiments on a GPU server with Nvidia GeForce RTX 3090 GPUs and CUDA 11.8. The server also features an Intel Xeon CPU E5-2678 v3 @ 2.50GHz and runs on Ubuntu 18.04.5 LTS. The code is implemented using Python 3.8 and PyTorch 1.8.1. The default LLM in GenLog is implemented by GPT-4, mainly because of the powerful semantic understanding and network search capabilities of GPT-4, which are important for understanding log semantics and generating appropriate log content. In addition, using ChatGPT as LLMs is popular in recent research [21, 60].

Following previous works [21, 22, 27], for experiments that may exhibit randomness, we repeat them three times and use the average of these results as the final performance.

4.3 Evaluation Metrics

As in prior research [22], we utilize the following metrics to evaluate log parser performance, including Group Accuracy (GA), Parsing Accuracy (PA), F1 Score of Group Accuracy (FGA), and F1 Score of Template Accuracy (FTA). To evaluate semantic log parsers, we introduce some specific metrics, they are Semantic Parsing Accuracy (SPA) at the message-level, Precision, Recall, and F1 Score at the category-level. Furthermore, for LLM-based parsers, we introduce cost metrics, including API Call Frequency (ACF), API Token Consumption (ATC), and Log Token Consumption (LTC).

4.3.1 Metrics for Semantic Log Parsing. To promote semantic log parsing, it is important to have proper metrics to accurately evaluate the performance of these parsers. Existing metrics are insufficient for such assessments. Thus, we design two types of semantic log parsing evaluation metrics for LogBase: message-level metrics – Semantic Parsing Accuracy (SPA), and category-level metrics – Precision, Recall, and F1 Score.

Semantic Parsing Accuracy (SPA): In semantic log parsing, a log message is considered “correctly parsed” if every token in the log is accurately identified as either a template or a semantic parameter [32]. Therefore, for the hierarchical semantic taxonomy in LogBase, we design SPA to measure the accuracy of semantic log parsing. Specifically, for the parsing result of a log message, we first check whether the constant tokens in the parsing result precisely match those in the ground truth (GT). Then, we check if the predicted semantic categories of parameter tokens in the parsing result match the corresponding categories in the GT. A log message is considered “correctly parsed” only when both conditions are met. SPA is a message-level metric, defined as the ratio of correctly parsed log entries to the total number of log messages, used to evaluate the overall performance of the semantic log parser.

Precision, Recall, F1 Score: In addition to SPA for overall performance evaluation, we employ Precision, Recall, and F1 Score to assess the model’s performance in identifying different semantic categories of logs. Each category is evaluated separately. Precision measures the parser’s accuracy in identifying relevant categories. Recall measures the parser’s completeness in identifying all relevant categories. The F1 Score represents the balance between precision and recall.

4.3.2 Cost Metrics for LLM-Based Parsers. With the rapid advancement of LLMs, LLM-based parsers have demonstrated remarkable performance. Serving as the core module, LLMs offer excellent results but come with high operational cost. Different designs of LLM-based parsers lead to various efficiencies in utilizing LLMs, which impacts operational expenses significantly. In industrial settings, where balancing performance with cost is an essential concern, thus it is crucial to have some specific metrics to evaluate this aspect. Existing LLM-based parsers operate by online APIs, incurring cost based on the number of tokens processed. To better assess the operational cost and efficiency of LLM-based parsers, we propose the following metrics:

API Call Frequency (ACF): This metric measures the number of API calls made per log message parsed. It reflects the efficiency of API utilization during the log parsing process. Lower ACF indicates fewer API calls are needed per log message, which suggests higher operational efficiency and typically results in lower cost.

API Token Consumption (ATC): ATC quantifies the number of tokens consumed per API call. This metric assesses the complexity and resource usage of each API interaction. Lower ATC indicates fewer tokens are used per call, suggesting higher cost-effectiveness for each interaction.

Table 2. Statistic comparison between LogBase and existing benchmarks. “Ann. Logs” means annotated logs, and “Sem. Ann.” stands for semantic annotations, indicating whether the benchmark has semantic annotations.

Benchmark	#Sources	#Templates	#Ann. Logs	Sem. Ann.
LogHub	16	1,313	32,000	✗
LogHub-2.0	14	3,488	50,416,618	✗
LogBase	130	85,300	1,250,661	✓

Log Token Consumption (LTC): This metric evaluates the number of tokens consumed per log message processed. It is directly related to the token cost of parsing each log message and is a crucial indicator of cost efficiency, especially when parsing a large number of complex logs. Higher efficiency (i.e., lower LTC) implies that less cost is required per log message.

5 Experimental Results

5.1 RQ1: What Are the Differences and Advantages of LogBase Compared to the Existing Benchmarks?

To answer RQ1, we check the differences between LogBase and existing datasets, and highlight the advantages of LogBase from four aspects: data richness and diversity, semantic annotation, evaluation metric, and construction methodology.

Data richness and diversity. As presented in Tab. 2, compared to LogHub-2.0 [22], LogBase has 9.3 times more sources of logs and 24.5 times more number of log templates, significantly surpassing the existing benchmark datasets in terms of log sources and template numbers. As described in Sec. 2.3.2, the logs in LogHub-2.0 are all system-level, whereas in real-world environments there are a significant number of application-level logs, leading to a substantial difference in log feature distribution. To address this issue, LogBase collects logs from a wide range of sources, including both application-level and system-level. As illustrated in Fig. 3, these sources are categorized into 6 domains based on the purposes of the open-source projects, covering various application scenarios. This feature makes LogBase a more realistic benchmark dataset.

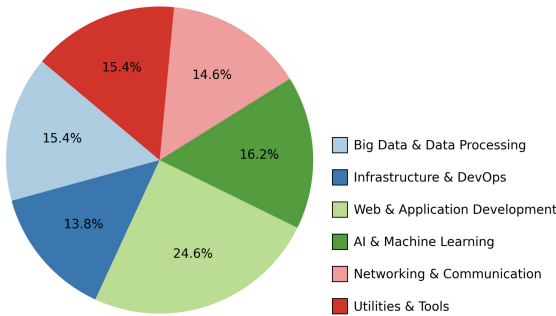


Fig. 3. Distribution of log sources in LogBase.

On the other hand, LogBase contains fewer annotated logs than LogHub-2.0 because we only generate 15 high-quality log messages per template in LogBase. We adopt this strategy because existing studies [21, 29] have demonstrated that deep learning-based log parsers require only a small number of annotated logs to effectively identify log templates. Therefore, generating a large number of logs for the same template is unnecessary. Notably, the number of annotated logs in LogBase is scalable. We can easily increase the number of logs via the GenLog framework.

Given that LogBase is quite different from the existing benchmarks in terms of data characteristics, i.e., the number of templates and the average number of parameters per template, we conduct a cross-dataset comparison to explore the impact of these differences. Since LogHub and LogHub-2.0 do

Table 3. The performance results of log parsers on different benchmarks. ‘-’ means data unavailable.

Method	LogHub				LogHub-2.0				LogBase			
	GA	PA	FGA	FTA	GA	PA	FGA	FTA	GA	PA	FGA	FTA
Drain	0.85	0.32	0.74	0.28	0.84	0.47	0.55	0.28	0.61	0.47	0.36	0.26
LogPPT	0.76	0.71	0.86	0.64	0.56	0.76	0.59	0.49	0.50	0.57	0.27	0.33
LILAC	-	-	-	-	0.93	0.84	0.92	0.81	0.66	0.56	0.54	0.42

not have semantic annotations, we choose three representative traditional parsers for performance comparison, including Drain (representing unsupervised parsers), LogPPT (representing supervised parsers), and LILAC (representing LLM-based parsers). For LogHub and LogHub-2.0, we use the performance reported in previous work [21, 22]. For LogBase, we follow the evaluation setup described in Sec. 4.2 to assess its performance.

The results in Tab. 3 show that LogBase introduces greater challenges compared to the existing benchmarks. All parsers show a decline in performance on LogBase. The most advanced LLM-based parser, LILAC, has the most significant performance decline, with PA dropping from 0.84 on LogHub-2.0 to 0.56 on LogBase (a 33% decrease), and FTA dropping from 0.81 to 0.42 (a 48% decrease). Similarly, LogPPT’s PA drops from 0.76 to 0.57 (a 25% decrease), highlighting the increased difficulty of LogBase. This decline is due to these parsers relying on a small number of examples for reference. On the simpler LogHub-2.0, these examples are enough, but with the enhanced complexity and template diversity in LogBase, they fail to generalize, resulting in a significant performance drop. This suggests that current parsers still struggle to perform effectively in complex environments, motivating the log parsing community to develop more advanced parsers based on LogBase.

Semantic Annotation. Existing benchmark datasets are designed specifically for general log parsing and do not support the training and evaluation of semantic log parsing models. LogBase, the first semantic log parsing benchmark dataset, features a hierarchical semantic taxonomy with fine semantic annotations for all logs, supporting the complete training-evaluation process for semantic log parsing models.

Evaluation Metric. The evaluation metrics of existing benchmark datasets are quite limited. LogHub only provides the GA metric, while LogHub-2.0 expands this to GA, PA, FGA, and FTA. In comparison, LogBase offers a more comprehensive set of evaluation metrics. In addition to general log parsing metrics such as GA, PA, ED, FGA, and FTA, it also includes metrics specifically designed for semantic log parsing. These include message-level metrics such as SPA and category-level metrics such as Precision, Recall, and F1 Score. Additionally, for the emerging LLM-based parsers, we propose three cost evaluation metrics: ACF, ATC, and LTC. These metrics enable a comprehensive and reliable evaluation of both general and semantic log parsers.

Construction Methodology. The construction of existing benchmark datasets involves collecting raw logs and manually annotating them with templates, which has two main flaws. First, the logs in existing benchmarks are mostly collected from large-scale servers, where the high cost hinders the continuous updates of these benchmarks. Many logs in benchmarks like LogHub and LogHub-2.0 are over a decade old, leading to discrepancies with modern logs as software systems evolve. Second, templates are annotated directly from logs without access to source code, introducing errors. Studies [27, 29] have shown that these benchmarks often contain incorrect annotations, which can mislead log parser training and evaluation.

LogBase addresses these issues above using the GenLog framework. It mines log templates and parameters directly from source code, ensuring accuracy. High-quality log data is generated

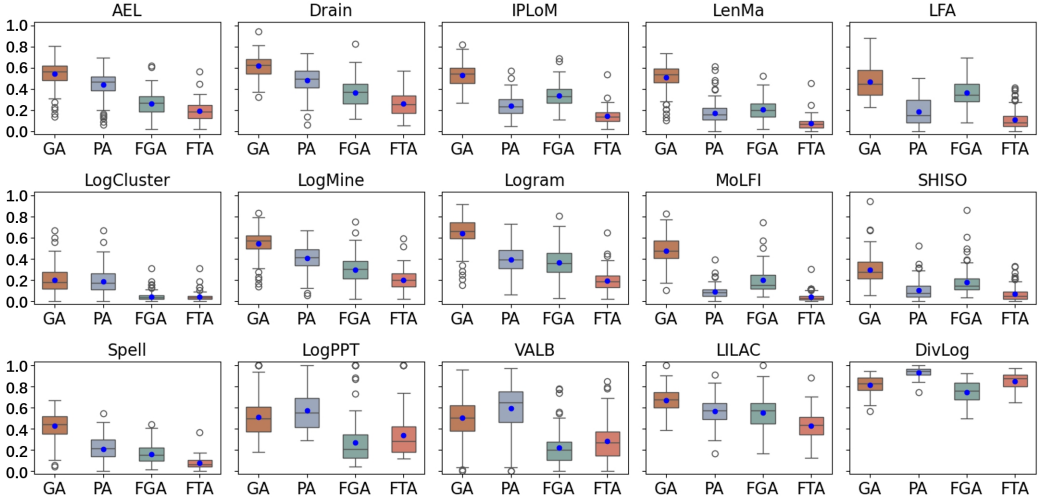


Fig. 4. Performance comparison of general log parsing on LogBase.

with LLM and CoT techniques, plus human feedback to enhance reliability. This automatic and cost-effective process allows for easy updates to LogBase, keeping it relevant to and consistent with the real-world environments.

5.2 RQ2: How Do Existing Parsers Perform in General Log Parsing on LogBase?

To answer RQ2, we evaluate fifteen log parsers on LogBase, including 11 unsupervised parsers: AEL [23], Drain [17], IPLoM [37], LenMa [45], LFA [40], LogCluster [51], LogMine [16], Logram [6], MoLFI [38], SHISO [39] and Spell [8], two supervised parsers: LogPPT [29] and VALB [32], and two LLM-based parsers: LILAC [21] and DivLog [60]. We measure their log parsing performance in terms of effectiveness, efficiency, and cost.

Effectiveness. We evaluate parser effectiveness on Java logs in LogBase using four metrics: GA, PA, FGA, and FTA. Due to space constraints, results are presented in box plots, showing the distribution from all log sources for each metric. As shown in Fig. 4, 1) unsupervised parsers perform the worst in terms of PA and FTA. The average PA for all unsupervised parsers is below 0.5, with six having an average FTA below 0.1. This is because these methods rely on statistical information and lack semantic understanding, leading to poor log template recognition precision. However, some unsupervised parsers outperform supervised ones in GA and FGA, as supervised parsers parse each log independently, which may result in different parsing outcomes for logs from the same template. This overlooks global information, leading to poor grouping performance. 2) LLM-based methods significantly outperform the others, with DivLog achieving the best results across all metrics, showing an average GA of 0.88, PA of 0.93, FGA of 0.74, and FTA of 0.84. Thanks to the comprehensive knowledge encapsulated in LLMs, LLM-based parsers understand log text semantics and precisely identify log templates with just a few annotated examples, without additional training.

Efficiency. We measure the average parsing speed of these parsers on LogBase to evaluate their efficiency. Among them, five unsupervised parsers parse over 1000 logs per second, with LFA being the fastest at 4907 logs per second. The slowest are the LLM-based parsers: LILAC and DivLog, parsing fewer than 10 logs per second. Such slow speed is due to the inherent processing time of LLMs and their reliance on OpenAI's online services, with network delay that further slows

their operational speeds. Considering the massive volume of logs in real-world environments, LLM-based parsers need to enhance their operational efficiency to meet practical demands.

Cost. Unsupervised parsers rely on statistical information for log parsing and consume only CPU resources. Supervised parsers require annotated data for model training and use GPU resources. These parsers can operate offline, incurring no additional cost beyond computing resources. In contrast, LLM-based parsers incur additional costs as they use API calls to online LLMs for log parsing, with fees based on resource consumption (i.e., tokens). We evaluate the costs of LILAC and DivLog using the ACF, ATC, and LTC metrics defined in Sec. 4. As presented in Tab. 4, LILAC and DivLog perform similarly in terms of ATC. Due to better performance in terms of ACF, LILAC has a lower overall cost in terms of LTC compared to DivLog. This is because LILAC uses a parsing cache with the LLM, allowing the parser to match unparsed logs with previously parsed templates, avoiding the need to call the LLM for each log, thus significantly reducing operational cost.

Table 4. Cost comparison between two LLM-based parsers.

LLM-based Parser	ACF	ATC	LTC
LILAC	0.07	265	18.6
DivLog	1	262	262

5.3 RQ3: How Do Existing Parsers Perform in Semantic Log Parsing on LogBase?

To answer RQ3, we evaluate existing semantic log parsers on Java logs in LogBase from both the message-level and the category-level perspectives. As detailed in Sec. 4.2, our evaluation includes performance comparison with VALB [32] and Bert-NER [25]. Following the setup of VALB, we randomly split the logs from each source in LogBase into training (20%), validation (20%), and testing (60%) data sets. We then independently train and evaluate VALB and Bert-NER on each source.

Table 5. Performance comparison across different semantic categories.

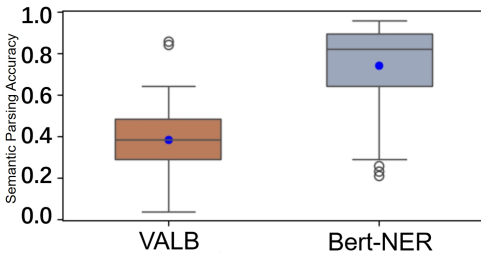


Fig. 5. Performance results of semantic log parsing on LogBase.

Category	Precision (%)		Recall (%)		F1 Score (%)	
	VALB	Bert-NER	VALB	Bert-NER	VALB	Bert-NER
OID	47.3	92.0	58.4	93.0	52.3	92.5
OBN	55.5	90.8	62.6	91.5	58.8	91.1
LOI	61.8	94.9	68.0	95.5	64.8	95.2
TID	45.6	81.4	4.1	72.0	7.6	76.4
SID	70.5	88.9	48.9	88.5	57.7	88.7
TPA	53.1	94.5	23.2	95.1	32.3	94.8
OBA	47.8	92.2	76.5	94.3	58.8	93.3
CRS	50.4	90.3	19.4	88.9	28.0	89.6
TDA	40.0	93.6	17.9	95.2	24.7	94.4
DCT	49.8	90.5	28.2	91.1	36.0	90.8
ERR	68.4	94.6	84.5	96.5	75.6	95.6
CMD	55.6	92.0	12.5	87.4	20.4	89.7
OBO	34.6	86.0	6.8	85.2	11.3	85.6
INR	31.6	84.7	15.4	76.1	20.7	80.2
VER	32.4	93.0	16.2	90.7	21.6	91.8
OTP	33.9	81.5	8.6	70.0	13.8	75.3
Average	48.6	90.1	34.5	88.2	36.5	89.1

Fig. 5 illustrates the semantic parsing performance of VALB and Bert-NER on LogBase, where each box encapsulates the distribution of the experimental results from all log sources. We can conclude: 1) the average performance of Bert-NER is better than VALB, the former has an average

SPA of 0.74, much higher than the latter's 0.38. 2) VALB is slightly more robust than Bert-NER. Among all these sources, the standard deviation of SPA for VALB is 0.15, while for Bert-NER, it is 0.2. Overall, Bert-NER significantly outperforms VALB in average performance. Although Bert-NER's performance is slightly unstable, the discrepancy is not significant.

Tab. 5 presents the performance of VALB and Bert-NER in recognizing different log semantic categories. By comparing their precision, recall, and F1 scores across different categories, we have the following observations: 1) Generally, Bert-NER outperforms VALB in recognizing all categories. Bert-NER has an average precision of 90.1%, recall of 88.2%, and F1 Score of 89.1% across all semantic categories, whereas VALB's average precision, recall, and F1 Score are 48.6%, 34.5%, and 36.5%, respectively. Bert-NER leads by a significant margin. 2) VALB shows a large performance gap across different semantic categories, with F1 scores for less common categories (e.g. TID, OBO, INR) being far below the average. In contrast, Bert-NER shows less performance variance across categories.

The results above indicate that Bert-NER achieves superior performance in semantic log parsing due to Bert's robust text feature representation. This capability arises from extensive pre-training on various natural language corpora, like Wikipedia and BookCorpus, enabling it to understand specific log terms. Additionally, Bert's sensitivity to textual context allows accurate extraction of semantic features from log tokens. Conversely, VALB has weaker text feature representation. Its reliance on static text embeddings and weaker context processing, especially GloVe's [43] out-of-vocabulary issues and BiLSTM's limitations in capturing long-distance dependencies, limits its performance. Thus, the key to semantic log parsing lies in the representations of log semantic features, and large pre-trained language models offer a promising direction.

5.4 RQ4: Are the Synthetic Logs Valid?

Unlike previous benchmarks, the logs in LogBase are generated using the GenLog framework. While this approach offers convenience, it raises concern about the quality of GenLog-generated logs. Specifically, whether the parameters in the synthetic logs can effectively simulate real environments.

To answer RQ4, we quantitatively evaluate the validity of synthetic logs in two ways: benchmark evaluation and human evaluation.

5.4.1 Benchmark Evaluation. For benchmark evaluation, we conduct experiments on the large-scale log dataset, *LogHub-2.0* [22], to assess the effectiveness of GenLog in generating quality data, thereby demonstrating the validity of data in LogBase. By using GenLog, we generate synthetic logs based on templates from *LogHub-2.0*, constructing a dataset called *Generated LogHub-2.0*. *Generated LogHub-2.0* matches *LogHub-2.0* in log templates and the number of logs, with the distinction that the parameters of the logs are generated. Since *LogHub-2.0* does not provide log parameter names and context code, we only use log templates as references for generation.

We verify the validity of synthetic logs by using the semantic log parsers VALB. The models are trained on two types of data: 1) real log data from the *LogHub-2.0* training set, and 2) synthetic log data from the *Generated LogHub-2.0*, which contains an equal number of synthesized logs with the same templates as the real training set. Then, the trained model is evaluated on the real *LogHub-2.0* test set. We follow the setup of VALB, randomly selecting 20% of the original data as the training set, 20% as the validation set, and 60% as the test set.

Tab. 6 shows the evaluation results, which indicate that the performance gap between the two types of training data is less than 6% in terms of the GA, PA, and SPA metrics. The performance of synthetic logs is very close to that of real logs. However, as for the FGA and FTA metrics, the gap is slightly larger, about 9%. This is because FGA and FTA are stricter template-level metrics, and in this experiment, the generation of synthetic logs relies solely on the templates. *LogHub-2.0* does not provide parameter names or context, and the synthetic logs are not corrected by human

feedback. These limitations restrict the performance of the GenLog framework, leading to lower performance in terms of FGA and FTA for the models trained on the synthetic logs. However, it should be noted that these limitations do not exist in the LogBase generation process, ensuring the GenLog framework produces high-quality logs.

Table 6. Performance comparison of VALB on different training data. The log templates in the *Generated LogHub-2.0* are the same as those in *LogHub-2.0*, while the parameters are generated by **GenLog**.

Parser	Training data	GA	PA	FGA	FTA	SPA
VALB	<i>LogHub-2.0</i>	0.76	0.89	0.64	0.61	0.76
	<i>Generated LogHub-2.0</i>	0.75	0.84	0.55	0.52	0.70

5.4.2 Human Evaluation. In addition to benchmark evaluation, we follow prior work [32, 42] to invite Ph.D students to conduct a human evaluation of the quality of the synthetic logs. Specifically, we randomly sample 15 logs per log template from both *LogHub-2.0* and *Generated LogHub-2.0*. For each template, this yields 15 synthetic logs and 15 corresponding real logs as references. We then divide these logs by template into three distinct groups (*Group₁*, *Group₂*, and *Group₃*) and distribute them to students responsible for operations and development in other teams in the company. Each group is independently assessed by four students.

For each log template, students evaluate the validity of the parameters in the synthetic logs by comparing them with those in the reference logs. A synthetic log is thought to be reliable if its parameters are semantically equivalent to and formatted consistently with the reference log parameters. Each log template is then rated based on the validity of its 15 synthetic logs, using a 1-5 scale from lowest to highest validity [42]. After evaluation, the scores for each template are averaged within each group to produce a final result. Finally, we analyze the evaluation results from the three groups.

Tab. 7 presents the evaluation results, showing that each group has at least 79% templates rated 4 or higher, and 90% rated 3 or higher. Given that the parameter names and contextual code are not available when generating the logs in *Generated LogHub-2.0*, which limits GenLog’s full performance, these evaluation results for *Generated LogHub-2.0* are acceptable, demonstrating the validity of the synthetic logs. In the generation of LogBase, templates, parameter names, and contextual code are accessible, allowing GenLog to perform at its full potential. Additionally, each synthetic log in LogBase undergoes human review to ensure its high quality.

Table 7. Results of human evaluation on the quality of synthetic logs.

Group	Number of templates	Rating distribution			
		[1,2]	[2,3]	[3,4]	[4,5]
<i>Group₁</i>	1163	3%	7%	9%	81%
<i>Group₂</i>	1163	1%	5%	11%	83%
<i>Group₃</i>	1162	2%	5%	14%	79%

Note that with the rapid advancement of LLMs, the use of LLMs to reduce manual efforts has drawn widespread attention. Recent work [2] has shown that in some software engineering annotation tasks, LLMs exhibit a level of consistency comparable to human labeling, including tasks highly relevant to our work, such as name-value inconsistencies [42] and semantic similarity [24] tasks. Synthetic data is also increasingly used in model training and fine-tuning. It is well known

Table 8. Log anomaly detection performance comparison among different training strategies. *Base* — using regular log templates; *VALB Sem.* — using log templates with complete names of semantic categories defined in VALB; *LogBase Sem.¹* — using log templates with semantic category abbreviations defined in LogBase; *LogBase Sem.²* — using log templates with complete names of semantic categories defined in LogBase.

Dataset	Training strategy	Precision	Recall	F1 Score
BGL	<i>Base</i>	92.2%	84.4%	88.1%
	<i>VALB Sem.</i>	91.9%	85.1%	88.4%
	<i>LogBase Sem.¹</i>	91.4%	84.4%	87.8%
	<i>LogBase Sem.²</i>	93.4%	87.1%	90.1%
HDFS	<i>Base</i>	95.6%	86.3%	90.7%
	<i>VALB Sem.</i>	97.2%	86.9%	91.8%
	<i>LogBase Sem.¹</i>	95.4%	87.1%	91.0%
	<i>LogBase Sem.²</i>	95.5%	91.0%	93.2%

that Llama 3.1 [13] and Gemma 2 [47] both use a significant proportion of synthetic data in their pre-training and supervised fine-tuning stages. In software engineering, researchers [5, 49, 55, 56] have shown the benefits of synthetic data, which helps solve the real-world data scarcity problem. Considering the long-standing difficulty of acquiring log data, synthetic log data will be a cost-effective alternative to support the research of large-scale log analysis.

6 Discussions

6.1 Effectiveness of Log Semantics on Downstream Tasks

VALB enhances the performance of traditional log anomaly detection methods such as SVM and clustering by incorporating log semantic abbreviations into log templates, preliminarily demonstrating that log semantics is able to support downstream tasks [32]. In recent years, deep learning-based methods have surpassed traditional approaches in log analysis tasks, and become the mainstream choice. However, in our practice, applying VALB's approach to more advanced deep anomaly detection models yields no performance improvement. We attribute this to the fact that deep models leverage log semantics for anomaly detection, whereas semantic abbreviations lack substantive semantic information, thus failing to provide enough useful input to the models. To explore how log semantics can support downstream log analysis tasks better, we conduct experiments on the log anomaly detection dataset proposed by the LogPAI team, which includes data from BGL and HDFS sources.

We use the advanced deep model LogFormer [15] in our experiment. LogFormer is a log anomaly detection framework based on transformers, which achieves good generalization capability through a two-stage training process involving pre-training and adapter-based tuning. For comparison, our experiment is conducted with four strategies using increasingly finer semantics as follows: 1) *Base*: Training and evaluating LogFormer with regular log templates that do not contain semantic labels, serving as the baseline model. 2) *VALB Sem.*: Replacing the wildcard <*> in regular log templates with complete names of the coarse-grained semantic categories defined in VALB, then using these data to train and evaluate LogFormer. 3) *LogBase Sem.¹*: Replacing the wildcard <*> in regular log templates with abbreviations of the fine-grained semantic categories (e.g. OID and OBN) defined in LogBase, then using these data to train and evaluate LogFormer. 4) *LogBase Sem.²*: Replacing the wildcard <*> in regular log templates with complete names of the fine-grained semantic categories (e.g. Object ID and Object Name) defined in LogBase, then using these data to train and evaluate LogFormer. All the results are presented in Tab. 8.

From Tab. 8, we can see that the model using templates with complete LogBase semantic category names achieves the highest F1 score, lifting F1 score by 2% on BGL and 2.5% on HDFS, compared to the baseline model using regular templates. Additionally, the inclusion of semantic information increases recall by 2.7% on BGL and by 4.7% on HDFS, helping the model better understand log content and identify hard-to-detect anomalies. This is the main reason for the performance improvement. The model using templates with LogBase semantic category abbreviations achieves similar F1 scores to the baseline, indicating that abbreviations alone are insufficient to enhance log semantics understanding. In contrast, complete semantic category names provide valuable information, boosting anomaly detection performance. When comparing the fine-grained semantics of LogBase to the coarse-grained semantics of VALB, although the model using templates with complete VALB semantic category names shows some improvement over the baseline, the increase in F1 scores with LogBase semantic categories is more pronounced. This indicates that the detailed semantic information provided by fine-grained categories is more beneficial to downstream tasks, which demonstrates the value of log semantics for downstream tasks and highlights the advantages of the fine-grained semantic categories defined in LogBase.

6.2 Synthesizing Logs with Open-Source LLMs

Open-source LLMs have made significant progress in recent years. Compared to closed-source models, open-source LLMs offer advantages in data privacy and cost. Here we evaluate the performance of a representative open-source model Llama3 in log synthesis. Specifically, we utilize Llama-3-8B-Instruct [13] and its supervised fine-tuned version, Llama-3-8B-Instruct_{SFT}, to generate logs on Python log sources. And we provide results from the closed-source model GPT-4 for comparison.

As presented in Tab. 9, the closed-source model GPT-4 achieves the best results. Llama-3-8B-Instruct performs poorly, with only a 14% success rate in the first round. After human feedback, its success rate increases to 62%, which is still unsatisfactory. The supervised fine-tuned model, Llama-3-8B-Instruct_{SFT}, shows much better performance, with a 55% success rate in the first round and 87% after feedback. Although there is still a considerable performance gap from the closed-source model, open-source LLMs remain a promising direction.

Table 9. The success rate of log synthesis using the GenLog framework with different base LLMs.

Base LLMs	Round-1 Success Ratio	Round-2 Success Ratio
GPT-4	73%	97%
Llama-3-8B-Instruct	14%	62%
Llama-3-8B-Instruct _{SFT}	55%	87%

6.3 Future Research

Although log generation and semantic log parsing are different tasks, they both rely on the model's understanding of log semantics. Future research should explore models capable of performing both tasks simultaneously, using multi-objective optimization to enhance the model's semantic understanding. This will improve the quality of log generation and the accuracy of semantic log parsing, thereby advancing log analysis as a whole.

7 Threats to Validity

7.1 Internal Threats

7.1.1 Reliability of Human Feedback. As a crucial component of human-enhanced log generation, human feedback is key to ensuring the quality of generated logs. To guarantee the reliability of this feedback, we invite four experienced engineers to independently review the raw content generated by the LLM. Their reviews and feedback are collated and compared. Consistent results are adopted, while discrepancies are discussed and resolved to reach a consensus among them.

7.1.2 Limited Evaluation Scope. Another concern is the limited scope of our evaluation for log parsing methods, due to the fact that some methods are not open-sourced. Nevertheless, we conduct a thorough evaluation of all available open-sourced methods, which include both unsupervised and supervised parsers, as well as two state-of-the-art LLM-based parsers. In addition to evaluating performance, we introduce three cost metrics that are essential for assessing LLM-based parsers, and evaluate these parsers based on the cost metrics. Notably, these LLM-based parsers have not been included in previous benchmark studies [22, 67].

7.2 Construct Threats

The semantics of log parameters are difficult to define with strict precision. For example, for a log parameter like ‘*Dafu_1*’, some may consider it an object ID, while others may regard it as an object name, and in fact, both interpretations could be acceptable. Although we can reach consensus on the semantics of such parameters through discussion, the definitions are still largely influenced by human subjectivity or bias. When reviewing logs generated by LLMs, we observe that for some logs, the semantic templates produced by LLMs do not match the human annotations. However, the generated content is still reasonable, suggesting that LLMs’ understanding of log semantics may differ from that of humans.

8 Conclusion

In this paper, we present LogBase, the first and largest semantic log parsing benchmark dataset to date, which comprises 130 log sources and 85,300 semantically annotated log templates. To achieve this, we develop the GenLog framework, an efficient method for dataset construction that integrates data collection, and human-enhanced log generation in a highly automated and low-cost process. We also design a set of comprehensive evaluation metrics for LogBase, covering both performance and cost aspects. With LogBase, we conduct a systematic evaluation of 15 existing methods. We believe this work will provide valuable data, reliable tools, and insightful perspectives for future semantic log parsing research.

9 Data Availability

The LogBase dataset is available in https://figshare.com/articles/dataset/LogBase_Dataset/28815620?file=53743838.

Acknowledgments

This work was supported by 2023 Alibaba Innovation Research (AIR) program. Chenbo Zhang and Wenying Xu contributed equally to this work.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).

- [2] Toufique Ahmed, Premkumar Devanbu, Christoph Treude, and Michael Pradel. 2024. Can LLMs Replace Manual Annotation of Software Engineering Artifacts? *arXiv preprint arXiv:2408.05534* (2024).
- [3] Anunay Amar and Peter C Rigby. 2019. Mining historical test logs to predict bugs and localize faults in the test logs. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. IEEE, 140–151.
- [4] Som Biswas. 2023. Role of ChatGPT in Computer Programming.: ChatGPT in Computer Programming. *Mesopotamian Journal of Computer Science* 2023 (2023), 8–16.
- [5] Federico Cassano, John Gouwar, Francesca Lucchetti, Claire Schlesinger, Carolyn Jane Anderson, Michael Greenberg, Abhinav Jangda, and Arjun Guha. 2023. Knowledge Transfer from High-Resource to Low-Resource Programming Languages for Code LLMs. *arXiv preprint arXiv:2308.09895* (2023).
- [6] Hetong Dai, Heng Li, Che-Shao Chen, Weiyi Shang, and Tse-Hsun Chen. 2020. Logram: Efficient log parsing using n n-gram dictionaries. *IEEE Transactions on Software Engineering* 48, 3 (2020), 879–892.
- [7] Anwesha Das, Frank Mueller, Charles Siegel, and Abhinav Vishnu. 2018. Dsh: deep learning for system health prediction of lead times to failure in hpc. In *Proceedings of the 27th international symposium on high-performance parallel and distributed computing*. 40–51.
- [8] Min Du and Feifei Li. 2016. Spell: Streaming parsing of system event logs. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*. IEEE, 859–864.
- [9] Min Du, Feifei Li, Guineng Zheng, and Vivek Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*. 1285–1298.
- [10] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [11] Qiang Fu, Jian-Guang Lou, Yi Wang, and Jiang Li. 2009. Execution anomaly detection in distributed systems through unstructured log analysis. In *2009 ninth IEEE international conference on data mining*. IEEE, 149–158.
- [12] Ying Fu, Meng Yan, Jian Xu, Jianguo Li, Zhongxin Liu, Xiaohong Zhang, and Dan Yang. 2022. Investigating and improving log parsing in practice. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1566–1577.
- [13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [14] Alex Graves and Jürgen Schmidhuber. 2005. Framewise phoneme classification with bidirectional LSTM and other neural network architectures. *Neural networks* 18, 5-6 (2005), 602–610.
- [15] Hongcheng Guo, Jian Yang, Jiaheng Liu, Jiaqi Bai, Boyang Wang, Zhoujun Li, Tieqiao Zheng, Bo Zhang, Junran Peng, and Qi Tian. 2024. Logformer: A pre-train and tuning pipeline for log anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 135–143.
- [16] Hossein Hamooni, Biplob Debnath, Jianwu Xu, Hui Zhang, Guofei Jiang, and Abdullah Mueen. 2016. Logmine: Fast pattern recognition for log analytics. In *Proceedings of the 25th ACM international on conference on information and knowledge management*. 1573–1582.
- [17] Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE international conference on web services (ICWS)*. IEEE, 33–40.
- [18] Shilin He, Qingwei Lin, Jian-Guang Lou, Hongyu Zhang, Michael R Lyu, and Dongmei Zhang. 2018. Identifying impactful service system problems via log analysis. In *Proceedings of the 2018 26th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 60–70.
- [19] Junjie Huang, Zhihan Jiang, Zhuangbin Chen, and Michael R Lyu. 2024. ULog: Unsupervised Log Parsing with Large Language Models through Log Contrastive Units. *arXiv preprint arXiv:2406.07174* (2024).
- [20] Yintong Huo, Yuxin Su, Cheryl Lee, and Michael R Lyu. 2023. Semparser: A semantic parser for log analytics. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 881–893.
- [21] Zhihan Jiang, Jinyang Liu, Zhuangbin Chen, Yichen Li, Junjie Huang, Yintong Huo, Pinjia He, Jiazhen Gu, and Michael R Lyu. 2024. Lilac: Log parsing using llms with adaptive parsing cache. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 137–160.
- [22] Zhihan Jiang, Jinyang Liu, Junjie Huang, Yichen Li, Yintong Huo, Jiazhen Gu, Zhuangbin Chen, Jieming Zhu, and Michael R. Lyu. 2024. A Large-Scale Evaluation for Log Parsing Techniques: How Far Are We? *arXiv:2308.10828* [cs.SE]
- [23] Zhen Ming Jiang, Ahmed E Hassan, Parminder Flora, and Gilbert Hamann. 2008. Abstracting execution logs to execution events for enterprise applications (short paper). In *2008 The Eighth International Conference on Quality Software*. IEEE, 181–186.
- [24] Marius Kamp, Patrick Kreutzter, and Michael Philippsen. 2019. SeSaMe: A data set of semantically similar Java methods. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 529–533.

- [25] Kamal Raj Kanakarajan. 2019. Bert-NER. <https://github.com/kamalkraj/BERT-NER>.
- [26] Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2312–2323.
- [27] Zanis Ali Khan, Donghwan Shin, Domenico Bianculli, and Lionel Briand. 2022. Guidelines for assessing the accuracy of log message template identification techniques. In *Proceedings of the 44th International Conference on Software Engineering*. 1095–1106.
- [28] Van-Hoang Le and Hongyu Zhang. 2022. Log-based anomaly detection with deep learning: How far are we?. In *Proceedings of the 44th international conference on software engineering*. 1356–1367.
- [29] Van-Hoang Le and Hongyu Zhang. 2023. Log parsing with prompt-based few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2438–2449.
- [30] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161* (2023).
- [31] Tsz-On Li, Wenxi Zong, Yibo Wang, Haoye Tian, Ying Wang, Shing-Chi Cheung, and Jeff Kramer. 2023. Nuances are the key: Unlocking chatgpt to find failure-inducing tests with differential prompting. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 14–26.
- [32] Zhenhao Li, Chuan Luo, Tse-Hsun Chen, Weiye Shang, Shilin He, Qingwei Lin, and Dongmei Zhang. 2023. Did we miss something important? studying and exploring variable-aware log abstraction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 830–842.
- [33] Qingwei Lin, Hongyu Zhang, Jian-Guang Lou, Yu Zhang, and Xuewei Chen. 2016. Log clustering based problem identification for online service systems. In *Proceedings of the 38th International Conference on Software Engineering Companion*. 102–111.
- [34] Yudong Liu, Xu Zhang, Shilin He, Hongyu Zhang, Liquan Li, Yu Kang, Yong Xu, Minghua Ma, Qingwei Lin, Yingnong Dang, et al. 2022. Uniparser: A unified log parser for heterogeneous log data. In *Proceedings of the ACM Web Conference 2022*. 1893–1901.
- [35] Jian-Guang Lou, Qiang Fu, Shenqi Yang, Ye Xu, and Jiang Li. 2010. Mining invariants from console logs for system problem detection. In *2010 USENIX Annual Technical Conference (USENIX ATC 10)*.
- [36] Qinyu Luo, Yining Ye, Shihao Liang, Zhong Zhang, Yujia Qin, Yaxi Lu, Yesai Wu, Xin Cong, Yankai Lin, Yingli Zhang, et al. 2024. RepoAgent: An LLM-Powered Open-Source Framework for Repository-level Code Documentation Generation. *arXiv preprint arXiv:2402.16667* (2024).
- [37] Adetokunbo AO Makanju, A Nur Zincir-Heywood, and Evangelos E Milios. 2009. Clustering event logs using iterative partitioning. In *Proceedings of the 15th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1255–1264.
- [38] Salma Messaoudi, Annibale Panichella, Domenico Bianculli, Lionel Briand, and Raimondas Sasnauskas. 2018. A search-based approach for accurate identification of log message formats. In *Proceedings of the 26th Conference on Program Comprehension*. 167–177.
- [39] Masayoshi Mizutani. 2013. Incremental mining of system log format. In *2013 IEEE International Conference on Services Computing*. IEEE, 595–602.
- [40] Meiyappan Nagappan and Mladen A Vouk. 2010. Abstracting log lines to log event types for mining software system logs. In *2010 7th IEEE Working Conference on Mining Software Repositories (MSR 2010)*. IEEE, 114–117.
- [41] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474* (2022).
- [42] Jibesh Patra and Michael Pradel. 2022. Nalin: learning from runtime behavior to find name-value inconsistencies in jupyter notebooks. In *Proceedings of the 44th International Conference on Software Engineering*. 1469–1481.
- [43] Jeffrey Pennington, Richard Socher, and Christopher D Manning. 2014. Glove: Global vectors for word representation. In *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*. 1532–1543.
- [44] Issam Sedki, Abdelwahab Hamou-Lhadj, Otmane Ait-Mohamed, and Naser Ezzati-Jivan. 2023. Towards a classification of log parsing errors. In *2023 IEEE/ACM 31st International Conference on Program Comprehension (ICPC)*. IEEE, 84–88.
- [45] Keichi Shima. 2016. Length matters: Clustering system log messages using length of words. *arXiv preprint arXiv:1611.03213* (2016).
- [46] Liang Tang, Tao Li, and Chang-Shing Perng. 2011. LogSig: Generating system events from raw textual logs. In *Proceedings of the 20th ACM international conference on Information and knowledge management*. 785–794.
- [47] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, and Cassidy Hardin et al. 2024. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv preprint arXiv:2408.00118* (2024).
- [48] Federico Tomassetti, N Smith, C Maximilien, and S Kirsch. 2021. JavaParser.

- [49] Yun-Da Tsai, Mingjie Liu, and Haoxing Ren. 2024. Code Less, Align More: Efficient LLM Fine-tuning for Code Generation with Data Pruning. *arXiv preprint arXiv:2407.05040* (2024).
- [50] Risto Vaarandi. 2003. A data clustering algorithm for mining patterns from event logs. In *Proceedings of the 3rd IEEE Workshop on IP Operations & Management (IPOM 2003)*(IEEE Cat. No. 03EX764). Ieee, 119–126.
- [51] Risto Vaarandi and Mauno Pihelgas. 2015. Logcluster-a data clustering and pattern mining algorithm for event logs. In *2015 11th International conference on network and service management (CNSM)*. IEEE, 1–7.
- [52] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [53] Xuheng Wang, Xu Zhang, Liqun Li, Shilin He, Hongyu Zhang, Yudong Liu, Lingling Zheng, Yu Kang, Qingwei Lin, Yingnong Dang, et al. 2022. SPINE: a scalable log parser with feedback guidance. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1198–1208.
- [54] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [55] Yeming Wen, Pengcheng Yin, Kensen Shi, Henryk Michalewski, Swarat Chaudhuri, and Alex Polozov. 2024. Grounding Data Science Code Generation with Input-Output Specifications. *arXiv preprint arXiv:2402.08073* (2024).
- [56] Jason Wu, Eldon Schoop, Alan Leung, Titus Barik, Jeffrey P Bigham, and Jeffrey Nichols. 2024. UICoder: Finetuning Large Language Models to Generate User Interface Code through Automated Feedback. *arXiv preprint arXiv:2406.07739* (2024).
- [57] Yifan Wu, Bingxu Chai, Siyu Yu, Ying Li, Pinjia He, Wei Jiang, and Jianguo Li. 2024. LogPTR: Variable-Aware Log Parsing with Pointer Network. *arXiv preprint arXiv:2401.05986* (2024).
- [58] Yifan Wu, Siyu Yu, and Ying Li. 2024. Log Parsing with Self-Generated In-Context Learning and Self-Correction. *arXiv preprint arXiv:2406.03376* (2024).
- [59] Yi Xiao, Van-Hoang Le, and Hongyu Zhang. 2024. Stronger, Faster, and Cheaper Log Parsing with LLMs. *arXiv preprint arXiv:2406.06156* (2024).
- [60] Junjielong Xu, Ruichun Yang, Yintong Huo, Chengyu Zhang, and Pinjia He. 2024. DivLog: Log Parsing with Prompt Enhanced In-Context Learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.
- [61] Siyu Yu, Pinjia He, Ningjiang Chen, and Yifan Wu. 2023. Brain: Log parsing with bidirectional parallel tree. *IEEE Transactions on Services Computing* (2023).
- [62] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No more manual tests? evaluating and improving chatgpt for unit test generation. *arXiv preprint arXiv:2305.04207* (2023).
- [63] Bo Zhang, Hongyu Zhang, Van-Hoang Le, Pablo Moscato, and Aozhong Zhang. 2023. Semi-supervised and unsupervised anomaly detection by mining numerical workflow relations from system logs. *Automated Software Engineering* 30, 1 (2023), 4.
- [64] Bo Zhang, Hongyu Zhang, Pablo Moscato, and Aozhong Zhang. 2020. Anomaly detection via mining numerical workflow relations from logs. In *2020 International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 195–204.
- [65] Shenglin Zhang, Ying Liu, Weibin Meng, Zhiling Luo, Jiahao Bu, Sen Yang, Peixian Liang, Dan Pei, Jun Xu, Yuzhi Zhang, et al. 2018. Prefix: Switch failure prediction in datacenter networks. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 2, 1 (2018), 1–29.
- [66] Xu Zhang, Yong Xu, Qingwei Lin, Bo Qiao, Hongyu Zhang, Yingnong Dang, Chunyu Xie, Xinsheng Yang, Qian Cheng, Ze Li, et al. 2019. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*. 807–817.
- [67] Jieming Zhu, Shilin He, Pinjia He, Jinyang Liu, and Michael R Lyu. 2023. Loghub: A large collection of system log datasets for ai-driven log analytics. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, 355–366.

Received 2024-10-31; accepted 2025-03-31